

PROTOCOL SPECIFICATION

Memcached 协议级压缩 标志协商规范

MC-COMPRESS-SPEC v1.0

引入 MAGIC 规范标识符实现确定性来源识别，定义 flags 字段结构化编码格式、压缩算法注册表、客户端行为约束及向后兼容策略。

2026 年 5 月

1 概述

1.1 背景与动机

Memcached 作为高性能分布式内存缓存系统，其协议设计中为每个存储的 item 预留了一个 32 位的 flags 字段，供客户端自定义使用。然而，memcached 社区从未对 flags 字段的位分配制定统一规范，导致各语言客户端 SDK 各自约定 flags 的使用方式。例如，PHP Memcached 扩展使用 flags 的第 1 位标记 zlib 压缩，而 Python 的 python-binary-memcached 使用不同的位约定。当不同语言的客户端访问同一 memcached 集群时，flags 解析冲突会导致解压失败甚至数据损坏。

随着业务系统中多语言技术栈的普及，以及 LZ4、Zstd、Snappy 等新型压缩算法在生产环境中的广泛部署，现有"各自约定"的模式已无法满足跨客户端互操作的需求。本规范旨在定义一套标准化的 flags 字段位分配方案和压缩协商协议，使任何遵循本规范的客户端实现都能正确识别并处理压缩数据，实现真正的跨语言、跨框架互操作。

本规范的核心改进在于引入了 MAGIC 字段作为规范标识符。当兼容客户端读取到一个 flags 值时，只需检查 MAGIC 字段即可确定该值是否由遵循本规范的客户端生成。这一机制使规范标识与传统数据的区分变得确定且高效，彻底消除了依赖启发式规则识别传统客户端数据的模糊性。同时，本规范对 CAR-ID 字段采用了 4 位精简设计，在满足当前及可预见未来需求的前提下，为应用自定义标记预留了充足的位空间。

1.2 规范目标

本规范的核心目标包括以下几个方面：第一，通过 MAGIC 字段建立确定性的规范标识机制，使兼容客户端能够无歧义地识别 flags 值的来源，彻底消除与传统客户端数据的混淆；第二，定义统一的 flags 字段位分配标准，明确每一位的语义和用途，消除不同客户端之间的歧义；第三，建立压缩算法注册表（Compression Algorithm Registry），为压缩算法的标识符分配提供权威的映射关系，并预留扩展机制以支持未来新算法的注册；第四，规定客户端在写入和读取压缩数据时的行为约束，包括压缩触发条件、flag 编码/解码流程、解压失败时的回退策略等；第五，确保向后兼容性，使采用本规范的新客户端能够与未采用本规范的旧客户端安全共存，不会导致数据损坏或服务中断。

1.3 术语定义

术语	定义
flags	memcached 协议中每个 item 附带的 32 位无符号整数，用于存储客户端自定义的元数据
item	memcached 中存储的一个完整键值对，包含 key、value、flags、cas 等属性
MC-FLAGS	本规范定义的 flags 字段结构化编码格式（见第 3 节）

MAGIC	MC-FLAGS 最高 4 位的规范标识符，固定值 0xA。用于确定性地判断 flags 值是否由本规范生成
CAR	Compression Algorithm Registry，压缩算法注册表（见第 4 节）
CAR-ID	压缩算法在 CAR 中注册的数字标识符，4 位无符号整数，取值范围 0x0-0xF
APP-FLAGS	MC-FLAGS 中供应用层自由使用的 16 位标记区域（bit 23-8），本规范不定义其语义
兼容客户端	实现了本规范的 memcached 客户端
传统客户端	未实现本规范、使用自有 flags 约定的 memcached 客户端

表 1: 术语定义

1.4 一致性要求

本规范使用 RFC 2119 定义的关键词来表示要求级别。"必须"(MUST)表示绝对要求；"不得"(MUST NOT)表示绝对禁止；"应当"(SHOULD)表示推荐但存在合理例外；"可以"(MAY)表示可选行为。声称符合本规范的实现必须满足所有 MUST 和 MUST NOT 级别的要求，否则不得声明符合本规范。

2 协议基础

2.1 memcached 协议中的 flags 字段

在 memcached 的二进制协议中，set/add/replace/cas 等存储命令的请求头包含一个 32 位的 flags 字段（offset 24-27，即第 6-7 字节）。在文本协议中，flags 以十进制数字形式出现在命令行中（如 "set key 0 3600 10" 中的 "0" 即为 flags）。该字段完全由客户端定义和使用，memcached 服务端对其不做任何解析或验证，仅原样存储并在 get/gets 响应中返回。

由于 flags 字段仅有 32 位，且需要在压缩标识、应用自定义标记等多种用途之间共享，因此需要精心设计位分配方案，避免位冲突。本规范的位分配设计遵循以下原则：最高位区域用于规范标识（MAGIC），确保确定性识别；压缩算法标识紧随 MAGIC 之后，便于快速定位；中间区域分配给应用自定义标记，提供充足的位空间以满足业务需求；最低区域保留给未来扩展。

2.2 二进制协议 vs 文本协议

memcached 支持两种协议：文本协议和二进制协议。二进制协议将 flags 作为请求/响应头中的固定字段传输，结构清晰，解析可靠。文本协议将 flags 作为存储命令的第二个参数以十进制数字传输，功能等价但缺乏结构化语义。本规范同时适用于两种协议。在文本协议中，兼容客户端必须将 MC-FLAGS 结构编码为一个 32 位无符号整数后再以十进制形式传输；在二进制协议中，MC-FLAGS 直接填入请求头的 flags 字段。

需要特别注意的是，本规范的 MAGIC 字段位于 bit 31-28，这意味着任何遵循本规范的 flags 值的高 4 位十六进制表示均为 0xA，对应的十进制值至少为 2,684,354,560（0xA0000000）。该值远超传统客户端使用的 flags 范围（通常为 0-65535），因此即便在文本协议中以十进制传输，也不会与传统 flags 值产生混淆。然而，某些旧版 memcached 服务端或客户端可能将 flags 限制为 16 位无符号整数（0-65535），兼容客户端在连接此类旧版服务端时，必须回退到不使用压缩的降级模式，并在连接建立时记录警告日志。

3 MC-FLAGS 字段编码规范

3.1 位分配总览

MC-FLAGS 将 32 位 flags 字段划分为以下四个区域。各区域的位范围和用途如后文所述。设计原则为：最高 4 位用于 MAGIC 规范标识，提供确定性的来源判断；压缩算法标识紧随 MAGIC 之后，便于在验证规范后立即获取压缩信息；中间 16 位分配给应用自定义标记，为业务逻辑提供充足的标记空间；最低 8 位保留给未来扩展。

位范围	位数	区域名称	用途
bit 31-28	4	MAGIC	规范标识符，固定值 0xA（二进制 1010）。若 MAGIC ≠ 0xA，则 flags 值不是由本规范生成的，兼容客户端必须按传统数据处理
bit 27-24	4	CAR-ID	压缩算法注册表标识符。0x0 = 未压缩，0x1-0xF = 已压缩且使用对应算法。4 位空间支持 16 种算法标识
bit 23-8	16	APP-FLAGS	应用自定义标记，供业务层自由使用，本规范不做定义。16 位空间提供 65536 种可能的值
bit 7-0	8	RESERVED	保留位，必须置 0。未来版本规范可能分配新用途

表 2: MC-FLAGS 字段位分配

图 1: MC-FLAGS 位布局图

MAGIC	CAR-ID	APP-FLAGS	RESERVED
28-31	24-27	8-23	0-7

3.2 MAGIC 字段 (bit 31-28)

MAGIC 字段是 MC-FLAGS 的核心机制，占据 bit 31 至 bit 28，共 4 位，固定值为 0xA（二进制 1010，十进制 10）。MAGIC 的作用是提供确定性的规范标识：当兼容客户端读取一个 flags 值时，首先检查其 MAGIC 字段。若 MAGIC = 0xA，则该 flags 值由遵循本规范的客户端生成，兼容客户端必须按照本规范定义的位布局进行解析；若 MAGIC ≠ 0xA，则该 flags 值来自传统客户端或使用其他编码方案，兼容客户端不得按照本规范解析，而应按照第 7 节规定的兼容性策略处理。

选择 0xA 作为 MAGIC 值基于以下考量：第一，二进制模式 1010 具有高度辨识度，与全零（传统客户端常见 flags 值）形成鲜明对比；第二，0xA 的十六进制表示简洁直观，任何以

十六进制查看的 flags 值，若最高位为 A，即可立即识别为本规范数据；第三，传统客户端使用的 flags 值通常为小整数（0、1、2、16 等），其 bit 31-28 始终为 0x0，不可能与 0xA 冲突——传统客户端的 flags 值从未使用过高位区域，因此本规范的 MAGIC 字段与所有已知的传统 flags 约定均不存在碰撞风险。

兼容客户端写入 item 时，必须将 MAGIC 字段设为 0xA。这一要求确保所有由兼容客户端生成的 flags 值都能被其他兼容客户端正确识别。兼容客户端不得（MUST NOT）使用 0xA 以外的 MAGIC 值，即使其他值在当前语境下似乎不会产生冲突。

3.3 CAR-ID 字段 (bit 27-24)

CAR-ID 字段占据 bit 27 至 bit 24，共 4 位，可表示 0x0-0xF 的数值（0-15）。CAR-ID 是 MC-FLAGS 中的核心信息字段，承担双重语义：当 CAR-ID = 0x0 时，表示 value 未经过压缩；当 CAR-ID 为 0x1-0xF 中的某个值时，表示 value 已被压缩，且该值即为压缩算法在 CAR 注册表中的标识符。这种设计将压缩状态和算法标识统一到单一字段中，无需独立的压缩使能位，简化了编码和解码逻辑。

4 位的 CAR-ID 空间提供 16 个标识符，这一设计基于以下分析：当前主流压缩算法仅有约 8 种（DEFLATE、LZ4、LZ4HC、Snappy、ZSTD、LZMA、Brotli 等），加上 NONE 标识，共需 8 个标识符；4 位空间除当前 8 个已注册算法外，还预留 8 个空位供未来扩展，在可预见的未来完全足够；即使未来压缩算法数量超过 16 种，也可以通过 RESERVED 字段扩展 CAR-ID 的位宽（例如将 RESERVED 的 bit 7 作为 CAR-ID 的扩展位，形成 5 位 CAR-ID），而无需修改已分配的标识符。

CAR-ID 的编码方式为无符号整数，bit 27 为最高位、bit 24 为最低位。在 flags 整数值中，CAR-ID 的值等于 $(\text{flags} \gg 24) \& 0xF$ 。当兼容客户端遇到未注册的 CAR-ID 值（即不在当前 CAR 注册表中的值）时，不得尝试解压，应当将 value 原样返回给调用方，并记录一条 WARNING 级别的日志。

3.4 应用自定义标记 (bit 23-8)

APP-FLAGS 字段占据 bit 23 至 bit 8，共 16 位，供应用层自由使用。16 位的 APP-FLAGS 为业务逻辑提供了 65536 种可能的标记值，这一宽裕的分配使得应用层可以在单一 flags 字段中编码丰富的业务语义，而无需与压缩相关的位发生冲突。本规范不对 APP-FLAGS 的语义做任何定义或约束，兼容客户端不得修改或覆盖此区域的值。

应用可以使用 APP-FLAGS 的某些位来标记数据是否来自主集群、是否允许被 LRU 淘汰、是否需要异步刷新、数据优先级、分区标识等业务语义。例如，使用 APP-FLAGS 的低 4 位编码数据优先级（0-15 级），使用 bit 12 标记是否来自主集群，使用 bit 13 标记是否需要异步刷新等。这种灵活的分配方式使应用层能够在不修改本规范的前提下，根据自身需求自由定义标记语义。

当兼容客户端写入 item 时，必须保留调用方传入的 APP-FLAGS 值不变，仅修改 bit 24-31（MAGIC、CAR-ID 相关位）和 bit 7-0（RESERVED 位清零）。如果调用方未提供 APP-FLAGS 值，兼容客户端应当将该区域置 0x0000。这一规则确保应用层对 APP-FLAGS 的

使用不会因压缩操作的介入而丢失。

3.5 保留位 (bit 7-0)

RESERVED 字段占据 bit 7 至 bit 0，共 8 位。当前版本规范 (v1.0) 中，此区域的所有位必须置 0。兼容客户端写入 item 时必须将 bit 7-0 清零；读取 item 时应当忽略此区域的值。未来版本的规范可能将保留位分配给新功能（如 CAR-ID 扩展位、压缩级别标识、校验和标记等），届时将遵循语义版本号规则进行升级。

选择将 RESERVED 放在最低字节而非其他位置，是基于以下考量：RESERVED 位于低位意味着其当前值（全零）不会影响 flags 整数值的高位部分，使 flags 值在十进制和十六进制表示中更加规整（最高位始终为 0xA）。此外，当未来需要扩展 CAR-ID 时，从 RESERVED 的高位 (bit 7、bit 6) 逐步借用比从 RESERVED 的低位借用更符合“从高位向低位扩展”的惯例。RESERVED 中的 bit 7 可作为 CAR-ID 的扩展位（形成 5 位 CAR-ID，支持 32 种算法），bit 6 可编码压缩级别的高位，bit 5-3 可用于未来校验和标记等用途。

3.6 flags 数值计算公式

MC-FLAGS 的各字段通过位运算组合为一个 32 位无符号整数。以下给出编码和解码的公式，兼容客户端的实现必须严格遵循这些公式。

编码（写入时构造 flags 值）：

```
flags = (MAGIC & 0xF) << 28  
| (CAR_ID & 0xF) << 24  
| (APP_FLAGS & 0xFFFF) << 8  
| (RESERVED & 0xFF)
```

解码（读取时解析 flags 值）：

```
magic = (flags >> 28) & 0xF  
car_id = (flags >> 24) & 0xF  
app_flags = (flags >> 8) & 0xFFFF  
reserved = flags & 0xFF
```

示例计算：

假设一个 value 使用 ZSTD 压缩 (CAR-ID = 0x5)，应用标记为 0x0000，保留位为 0x00，则 flags 值计算如下：MAGIC = 0xA，CAR-ID = 0x5，APP-FLAGS = 0x0000，RESERVED = 0x00。编码结果为 $flags = (0xA \ll 28) | (0x5 \ll 24) | (0x0000 \ll 8) | 0x00 = 0xA5000000$ ，十进制为 2,764,800,000。该值的十六进制表示 0xA5000000 中，最高位 A 即为 MAGIC 标识，5 为 ZSTD 的 CAR-ID，中间 0000 为 APP-FLAGS，末尾 00 为 RESERVED。

4 压缩算法注册表（CAR）

4.1 注册表设计原则

压缩算法注册表（Compression Algorithm Registry, CAR）是本规范定义的算法标识符与压缩算法实现之间的权威映射关系。CAR 的设计遵循以下原则：第一，确定性——同一 CAR-ID 在所有兼容客户端中必须映射到同一算法，不允许出现歧义；第二，稳定性——已分配的 CAR-ID 不会回收或重新分配给其他算法；第三，可扩展性——未分配的 CAR-ID 可通过规范的修订版本进行分配，无需修改现有实现；第四，可互操作——对于某个 CAR-ID，所有兼容客户端必须支持对应算法的解压，但压缩是可选的。

4.2 已注册算法

CAR-ID	算法名称	压缩级别要求	说明
0x0	NONE	N/A	无压缩。value 以原始字节串存储。CAR-ID = 0x0 是"未压缩"的唯一定义标识，不得用于标记已压缩的 value
0x1	DEFLATE	默认级别 6	RFC 1951 定义的 DEFLATE 算法，兼容 zlib 格式（RFC 1950），即 zlib header + DEFLATE + Adler-32 校验
0x2	LZ4	默认级别 1	LZ4 Block 格式。高速压缩与解压，适用于对延迟敏感的实时缓存场景
0x3	LZ4HC	默认级别 9	LZ4 高压压缩变体。压缩速度较 LZ4 慢，但压缩比更高，解压速度与 LZ4 相同
0x4	SNAPPY	默认级别	Google Snappy 格式。设计目标是极高吞吐量，压缩比适中，适用于大规模数据缓存
0x5	ZSTD	默认级别 3	Facebook Zstandard 算法。兼顾压缩比和速度，支持 1-22 级压缩，推荐级别 3 作为默认
0x6	LZMA	默认级别 6	LZMA/XZ 算法。极高压缩比但压缩速度极慢，仅适用于写入频率极低且存储空间极度紧张的场景
0x7	BROTLI	默认级别 4	Brotli 压缩算法。在 Web 内容压缩领域表现优异，压缩比通常优于 DEFLATE
0x8-0xF	RESERVED	N/A	保留。未分配。8 个空位供未来新算法注册，若超过 16 种算法则需要通过 RESERVED 字段扩展 CAR-ID 位宽

表 3：压缩算法注册表（CAR）

4.3 压缩级别协商

本规范不将压缩级别（compression level）编码到 MC-FLAGS 中，原因是：flags 空间有限，且压缩级别对解压方完全透明（同一算法的不同压缩级别产生相同格式的输出，解压流程完全一致）。因此，压缩级别由写入方自行决定，本规范仅推荐各算法的默认压缩级别（见表 3），但不做强制要求。

兼容客户端的实现应当允许调用方在写入时指定压缩级别。如果调用方未指定，兼容客户端应当使用表 3 中列出的默认级别。对于解压操作，兼容客户端不需要也无法获知压缩级别，因为解压逻辑与压缩级别无关。未来版本的规范可能利用 RESERVED 字段中的某些位来编码压缩级别信息，但当前版本不提供此功能。

4.4 新算法注册流程

当需要注册新的压缩算法时，应遵循以下流程：首先，注册申请人向本规范的维护组织提交注册申请，包含算法名称、参考实现链接、CAR-ID 建议值（可选）、压缩/解压性能基准数据等材料。其次，维护组织评审申请材料，重点评估算法的成熟度、跨平台可用性、与现有算法的差异化程度。评审通过后，维护组织分配下一个可用的 CAR-ID 并更新注册表。最后，新 CAR-ID 在规范修订版本中正式发布，各兼容客户端实现可在后续版本中添加支持。

当前 4 位 CAR-ID 空间中，0x8-0xF 共 8 个标识符处于保留状态。当这些保留标识符全部分配完毕后，新算法的注册需要通过 RESERVED 字段扩展 CAR-ID 位宽。具体的扩展方案将在未来版本的规范中定义，但基本思路是将 RESERVED 的最高位（bit 7）作为 CAR-ID 的扩展位，使 CAR-ID 从 4 位扩展为 5 位（支持 32 种算法），同时保持已分配的 4 位 CAR-ID 值不变。这种向后兼容的扩展策略确保了 4 位 CAR-ID 空间不会成为瓶颈。

在新 CAR-ID 正式发布之前，兼容客户端不得使用未注册的 CAR-ID 值。如果兼容客户端遇到未知的 CAR-ID，必须按照第 6 节规定的回退策略处理。

5 客户端行为规范

5.1 写入流程（SET/ADD/REPLACE）

当兼容客户端执行写入操作时，必须遵循以下流程。每一步均为 MUST 级别的要求，除非明确标注为 SHOULD 或 MAY。

步骤	操作	约束级别
1	设置 MAGIC = 0xA	MUST
2	评估 value 的字节长度是否超过压缩阈值（compression_threshold）。默认阈值为 1024 字节，可由调用方配置	MUST
3a	若 value 长度低于阈值：设置 CAR-ID = 0x0（未压缩），value 不压缩	MUST
3b	若 value 长度达到或超过阈值：使用配置的压缩算法对 value 进行压缩。压缩后若长度反而增大（如对已压缩数据再压缩），则回退到步骤 3a	MUST
3c	若压缩成功：设置 CAR-ID = 压缩算法的 CAR-ID（0x1-0xF）	MUST
4	保留或清零 APP-FLAGS（bit 23-8），不得覆盖调用方传入的值	MUST
5	将 RESERVED（bit 7-0）清零	MUST
6	按照第 3.6 节的编码公式构造 flags 值，将 flags 和压缩后的 value 一并发送给 memcached	MUST

表 4：写入流程规范

5.2 读取流程（GET/GETS）

当兼容客户端执行读取操作时，必须遵循以下流程。本规范的读取流程以 MAGIC 校验为入口，实现了确定性的规范识别，无需依赖启发式规则判断 flags 值的来源。

步骤	操作	约束级别
1	从 memcached 响应中获取 flags 值和 value 字节串	MUST
2a	提取 MAGIC = (flags >> 28) & 0xF。若 MAGIC = 0xA，进入步骤 3（本规范数据）	MUST
2b	若 MAGIC ≠ 0xA，该 flags 值不是由本规范生成的。按照第 7 节规定的传统客户端兼容策略处理	MUST
3a	解析 CAR-ID = (flags >> 24) & 0xF。若 CAR-ID = 0x0：value 未压缩，直接进入步骤 5	MUST
3b	若 CAR-ID ≠ 0x0 且为已知 CAR-ID：使用对应算法解压 value，进入步骤 5	MUST
3c	若 CAR-ID ≠ 0x0 且为未知 CAR-ID：执行回退策略（见第 6 节）	MUST

4	提取 APP-FLAGS = (flags >> 8) & 0xFFFF。将解压后的 value（或原始 value）连同 app_flags 等元信息返回给调用方	MUST
---	--	------

表 5：读取流程规范

5.3 压缩阈值配置

压缩阈值（compression_threshold）决定了 value 的字节长度达到多少时才触发压缩。本规范规定默认压缩阈值为 1024 字节，兼容客户端应当支持调用方通过配置参数覆盖此默认值。将阈值设为 0 表示对所有 value 均尝试压缩；将阈值设为无限大（如 UINT32_MAX）表示禁用压缩功能。

选择 1024 字节作为默认阈值基于以下考量：小于 1KB 的 value 通常压缩收益极低（压缩后体积减少不足 30%），而压缩/解压的 CPU 开销却不可忽略；1KB 以上的 value 压缩收益显著（JSON/文本类数据通常可压缩 60-80%），CPU 开销物有所值。兼容客户端可以（MAY）根据数据特征自适应调整阈值，但不得低于 128 字节。

5.4 多算法支持要求

兼容客户端的实现必须（MUST）支持 DEFLATE（CAR-ID 0x1）算法的解压，因为 DEFLATE 是最广泛部署的压缩算法，且是现有大多数 memcached 客户端 SDK 的默认压缩算法。对于 CAR 中的其他算法，兼容客户端应当（SHOULD）支持 LZ4（CAR-ID 0x2）和 ZSTD（CAR-ID 0x5）的解压，因为这两种算法在性能和压缩比方面均显著优于 DEFLATE，已成为行业新标准。

兼容客户端可以（MAY）选择性地支持其他算法的解压。对于写入操作使用的压缩算法，兼容客户端可以（MAY）支持任意 CAR 中注册的算法，但推荐优先使用 LZ4（低延迟场景）或 ZSTD（均衡场景）。兼容客户端必须（MUST）允许调用方通过配置参数指定写入时使用的压缩算法。

6 回退与容错策略

6.1 MAGIC 校验失败处理

当兼容客户端在读取流程中发现 $MAGIC \neq 0xA$ 时，表示该 flags 值不是由遵循本规范的客户端生成的。这种情况下，兼容客户端不得按照本规范的位布局解析 flags 值，而必须按照第 7 节规定的传统客户端兼容策略进行处理。MAGIC 校验是本规范中区分规范数据与传统数据的第一道防线，其判断结果直接决定后续的解析路径。

MAGIC 值	判断	处理方式
0x0	传统客户端数据 (flags 为小整数)	按照传统 flags 约定识别规则处理 (见第 7.2 节)
0xA	本规范数据	按照本规范位布局解析
其他值	未知编码方案	将 value 原样返回给调用方，记录 WARNING 日志

表 6: MAGIC 校验失败处理策略

6.2 未知 CAR-ID 回退

当兼容客户端在读取流程中通过 MAGIC 校验确认 flags 值来自本规范，但发现 CAR-ID 不在本地支持的算法列表中时，必须按照以下优先级顺序执行回退策略。每一步尝试失败后进入下一步，不得跳过。

优先级	回退策略	说明
1 (最高)	尝试动态加载对应算法的解压库	如果兼容客户端支持插件化算法注册机制，可以尝试在运行时加载对应的解压库。加载成功则正常解压，加载失败则进入下一优先级
2	将压缩 value 原样返回给调用方	兼容客户端将 value 作为原始字节串返回，同时在返回对象中标记 <code>is_compressed = true</code> 和 <code>car_id</code> ，让调用方自行处理解压
3 (最低)	记录错误日志并返回 MISS	如果调用方不接受压缩数据（即要求客户端必须返回未压缩数据），则兼容客户端记录 ERROR 级别日志，并向调用方返回缓存未命中 (MISS)

表 7: 未知 CAR-ID 回退策略

6.3 解压失败处理

当兼容客户端成功识别了 CAR-ID 但解压过程中出现错误（如数据损坏、校验和不匹配、内存不足等）时，必须 (MUST) 执行以下操作：记录 ERROR 级别日志，包含 key、CAR-ID、

错误类型和错误消息；向调用方返回缓存未命中（MISS）而非抛出异常，以避免单个损坏 item 导致整个请求链路崩溃；如果兼容客户端实现了指标上报机制，应当（SHOULD）递增 "decompress_error" 计数器，以支持监控和告警。

兼容客户端不得（MUST NOT）将损坏的解压数据返回给调用方。如果解压过程返回了部分数据但校验失败，必须丢弃所有已解压的数据。这一规则同样适用于 DEFLATE 算法的 Adler-32 校验、ZSTD 的 XXH64 校验等。解压失败属于不可恢复错误，任何试图容错返回部分数据的行为都可能导致上层应用出现难以排查的数据不一致问题。

6.4 压缩后体积增大回退

某些数据类型（如已压缩的图片、视频片段、加密数据等）在压缩后体积反而会增大。当兼容客户端在写入流程中发现压缩后的 value 长度大于原始 value 长度时，必须（MUST）放弃压缩，改为存储原始 value，并将 CAR-ID 置 0x0。这一回退策略确保 memcached 中永远不会存储"压缩后反而更大"的数据，避免内存和带宽的浪费。

兼容客户端可以（MAY）实现"压缩收益阈值"机制：当压缩后的体积减少比例低于某个阈值（如低于 10%）时，同样放弃压缩。这种策略可以在压缩收益微乎其微时节省解压的 CPU 开销。压缩收益阈值与压缩阈值是两个独立的配置参数：压缩阈值决定是否尝试压缩，而压缩收益阈值决定压缩后是否接受结果。

7 向后兼容性

7.1 与传统客户端共存

在实际生产环境中，兼容客户端和传统客户端通常需要访问同一 memcached 集群。本规范通过以下机制确保两类客户端的安全共存。由于本规范是首个版本，不存在与旧版规范的迁移问题，兼容性挑战完全来源于传统客户端（即未实现本规范的客户端）的共存需求。

第一，写入隔离。兼容客户端写入的数据，传统客户端读取时可能无法正确解压（因为传统客户端不理解 MC-FLAGS 的编码方式）。但这不会导致数据损坏——传统客户端将 value 原样返回给调用方，调用方可能看到乱码，但不会引发崩溃或数据丢失。为缓解此问题，建议采用 key 前缀隔离策略：兼容客户端写入的 key 使用 "mc:" 前缀，传统客户端写入的 key 不使用前缀。这样可以避免两类客户端误读对方的数据。

第二，渐进迁移。建议按以下顺序执行迁移：先部署兼容客户端的读取能力（即兼容客户端能够正确读取传统客户端写入的 zlib 压缩数据），再逐步切换写入路径到 MC-FLAGS 编码格式，最后下线传统客户端。迁移期间，兼容客户端必须同时支持 MC-FLAGS 编码格式和传统客户端的 flags 约定。由于 MAGIC 字段 (0xA) 位于 bit 31-28，与传统客户端使用的低位 flags 值完全不重叠，兼容客户端可以通过 MAGIC 校验无缝区分两类数据，实现零停机迁移。

7.2 传统 flags 约定识别

为支持渐进迁移，兼容客户端需要能够识别传统客户端的 flags 编码方式。当兼容客户端读取到 $MAGIC \neq 0xA$ 的 flags 值时，应使用以下启发式规则判断 flags 值的来源并选择正确的处理方式。这些规则应当 (SHOULD) 在兼容客户端中实现。

条件	判断	处理方式
$flags = 0$	传统客户端（未压缩）	value 未压缩，直接使用
$flags = 1$ （仅 bit 0 = 1，其余位为 0）	传统客户端（zlib 压缩，如 PHP 扩展）	尝试 zlib 解压；失败则视为原始数据
$flags = 2$ （仅 bit 1 = 1）	传统客户端（快速压缩标记）	尝试 zlib 解压；失败则视为原始数据
$0 < flags < 65536$ 且 $MAGIC \neq 0xA$	传统客户端（自定义标记）	视为未压缩数据，将 flags 和 value 原样返回给调用方
$flags \geq 65536$ 且 $MAGIC \neq 0xA$	未知编码方案	记录 WARNING 日志，将 value 原样返回给调用方

表 8：传统 flags 约定识别规则

8 安全性考量

8.1 压缩炸弹

压缩炸弹 (zip bomb) 是一种恶意构造的压缩数据，其解压后的体积远超压缩前的体积。例如，一个 1KB 的压缩数据解压后可能膨胀为 1GB 甚至更大，导致兼容客户端内存耗尽或服务崩溃。本规范要求兼容客户端必须实现以下防护措施：设置解压后的最大体积限制 (decompression_limit)，默认值为原始 value 大小的 100 倍或 64MB 中的较小值；当解压后的数据体积超过此限制时，立即终止解压操作，丢弃已解压的数据，并按照第 6.3 节的规定向调用方返回 MISS。

兼容客户端应当 (SHOULD) 在解压过程中持续监控已解压的数据量，而非等到解压完成后再检查体积。这意味着兼容客户端需要使用流式解压接口 (而非一次性解压接口)，在每次读取解压数据后检查累计体积是否超过限制。此外，兼容客户端应当 (SHOULD) 限制单个解压操作的 CPU 时间，以防止恶意构造的高复杂度压缩数据消耗过多的 CPU 资源。

8.2 数据完整性

压缩算法的内置校验机制为数据完整性提供了基本保障。例如，DEFLATE (zlib 格式) 使用 Adler-32 校验和，ZSTD 使用 XXH64 校验和，LZMA/XZ 使用 CRC-64 校验和。当解压过程中校验失败时，兼容客户端必须按照第 6.3 节的规定丢弃所有已解压的数据，不得将损坏的数据返回给调用方。

需要注意的是，并非所有压缩算法都提供校验机制。例如，LZ4 Block 格式和 Snappy 格式本身不包含校验和。当使用这些算法时，兼容客户端可以 (MAY) 在应用层实现额外的完整性校验 (如在 value 的头部或尾部附加 CRC-32 校验值)，但本规范不做强制要求。未来版本的规范可能利用 RESERVED 字段中的某些位来编码轻量级校验标记，以增强数据完整性保障。

8.3 算法降级攻击

算法降级攻击是指攻击者通过篡改 flags 值中的 CAR-ID 字段，使兼容客户端误认为 value 使用了较弱的压缩算法或未压缩，从而绕过压缩相关的安全检查。例如，攻击者可以将 CAR-ID 从 0x5 (ZSTD) 修改为 0x0 (NONE)，使兼容客户端跳过解压步骤，直接将压缩后的字节串当作原始数据返回给调用方，可能导致调用方解析异常或信息泄露。

本规范的 MAGIC 字段为检测此类攻击提供了基础：由于 MAGIC 字段位于 flags 的最高 4 位，攻击者要修改 CAR-ID (bit 27-24)，必须保持 MAGIC (bit 31-28) 为 0xA 才能通过校验。然而，本规范并未提供 flags 值的密码学完整性保护，因此在面对有能力直接修改 memcached 服务端内存的攻击者时，flags 值仍可能被篡改。对于需要强完整性保障的场景，建议在应用层对 value 实现附加的 HMAC 或数字签名机制。

此外，兼容客户端应当（SHOULD）在解压后验证数据的合理性（如检查数据长度、格式头、校验和等），以检测潜在的算法降级攻击。如果解压后的数据明显不符合预期格式（例如，CAR-ID 标识为 ZSTD 压缩，但解压后的数据不是有效的 ZSTD 格式），兼容客户端应当记录 WARNING 日志，并按照第 6.3 节的规定返回 MISS。

9 参考实现伪代码

9.1 编码（写入时）

以下伪代码展示如何在写入时构造 MC-FLAGS 值。该函数接收压缩算法标识符、应用自定义标记等参数，返回编码后的 32 位 flags 值。编码过程严格遵循第 3.6 节定义的公式，确保各字段正确映射到对应的位区域。

```
function encode_flags(car_id, app_flags):
    MAGIC = 0xA
    RESERVED = 0x00
    flags = (MAGIC & 0xF) << 28
    | (car_id & 0xF) << 24
    | (app_flags & 0xFFFF) << 8
    | (RESERVED & 0xFF)
    return flags
```

9.2 解码（读取时）

以下伪代码展示如何在读取时解析 MC-FLAGS 值。该函数接收 32 位 flags 值，首先校验 MAGIC 字段，若通过则提取 CAR-ID 和 APP-FLAGS；若 MAGIC 校验失败，则按照传统客户端数据处理。解码过程严格遵循第 3.6 节定义的公式，确保各字段从正确的位区域中提取。

```
function decode_flags(flags):
    magic = (flags >> 28) & 0xF
    if magic != 0xA:
        // 不是本规范数据，按传统客户端处理
        return handle_legacy_flags(flags)
    car_id = (flags >> 24) & 0xF
    app_flags = (flags >> 8) & 0xFFFF
    reserved = flags & 0xFF
    return {magic, car_id, app_flags, reserved}
```

9.3 完整写入流程

以下伪代码展示完整的写入流程，包括压缩阈值判断、压缩执行、CAR-ID 设置、flags 编码等步骤。该流程严格遵循第 5.1 节规定的写入流程规范，确保每一步操作的约束级别符合要求。

```
function mc_set(key, value, app_flags=0x0000,
    compression_threshold=1024,
    compression_algo=ZSTD):
    car_id = 0x0 // 默认未压缩
    stored_value = value
    // 评估是否需要压缩
    if len(value) >= compression_threshold:
        compressed = compress(value, compression_algo)
```

```
if len(compressed) < len(value):
    car_id = compression_algo.car_id
    stored_value = compressed
    // 编码 flags
    flags = encode_flags(car_id, app_flags)
    // 发送到 memcached
    memcached.set(key, flags, stored_value)
```

9.4 完整读取流程

以下伪代码展示完整的读取流程，包括 MAGIC 校验、CAR-ID 解析、解压执行、APP-FLAGS 提取等步骤。该流程严格遵循第 5.2 节规定的读取流程规范，确保 MAGIC 校验失败时正确回退到传统客户端处理，未知 CAR-ID 时正确执行回退策略。

```
function mc_get(key):
    flags, value = memcached.get(key)
    if flags is None:
        return MISS
    decoded = decode_flags(flags)
    if decoded.is_legacy:
        return handle_legacy_data(flags, value)
    // 本规范数据
    if decoded.car_id == 0x0:
        // 未压缩
        return {value, decoded.app_flags, compressed=False}
    elif decoded.car_id in SUPPORTED_ALGORITHMS:
        // 已知算法，执行解压
        decompressed = decompress(value, decoded.car_id)
        return {decompressed, decoded.app_flags, compressed=True}
    else:
        // 未知 CAR-ID，执行回退
        return handle_unknown_car_id(decoded.car_id, value,
            decoded.app_flags)
```